

EstateFlow

Dokumentacja Techniczna

Aplikacja Internetowa do Zarządzania Nieruchomościami

Stack: MongoDB · Express · React · Node.js
2024

1. Opis projektu

EstateFlow to pełnostackowa aplikacja internetowa do zarządzania ogłoszeniami nieruchomości, zbudowana w oparciu o stos MERN (MongoDB, Express, React, Node.js). Umożliwia użytkownikom przeglądanie ofert sprzedaży i wynajmu, tworzenie i zarządzanie własnymi ogłoszeniami oraz bezpośredni kontakt z właścicielami nieruchomości.

Aplikacja obsługuje:

- Przeglądanie aktualnych ofert, nieruchomości na wynajem i sprzedaż na stronie głównej
- Zaawansowane wyszukiwanie z filtrami (typ, udogodnienia, oferty specjalne, sortowanie)
- Pełne zarządzanie ogłoszeniami — tworzenie, edycja, usuwanie z przesyłaniem zdjęć
- Autoryzację przez email/hasło oraz Google OAuth (Firebase)
- Persystencję sesji użytkownika (Redux Persist)
- Bezpieczne API oparte na JWT z sesjami w ciasteczkach HTTP-only
- Zarządzanie profilem — przesyłanie awatara do Firebase Storage
- Wdrożenie w Dockerze — jeden serwer Express obsługuje API i aplikację React

2. Stos technologiczny

Warstwa	Technologia
Frontend	React 18, Vite 4, Tailwind CSS 3
Routing	React Router DOM v6
Stan aplikacji	Redux Toolkit 1.9, Redux Persist
Komponenty UI	Swiper (karuzela), React Icons
Backend	Node.js 20, Express 4
Baza danych	MongoDB 7 (Mongoose ODM)
Autoryzacja	JWT (HTTP-only cookie), bcryptjs
OAuth	Firebase Authentication (Google)
Pliki	Firebase Storage
Konteneryzacja	Docker

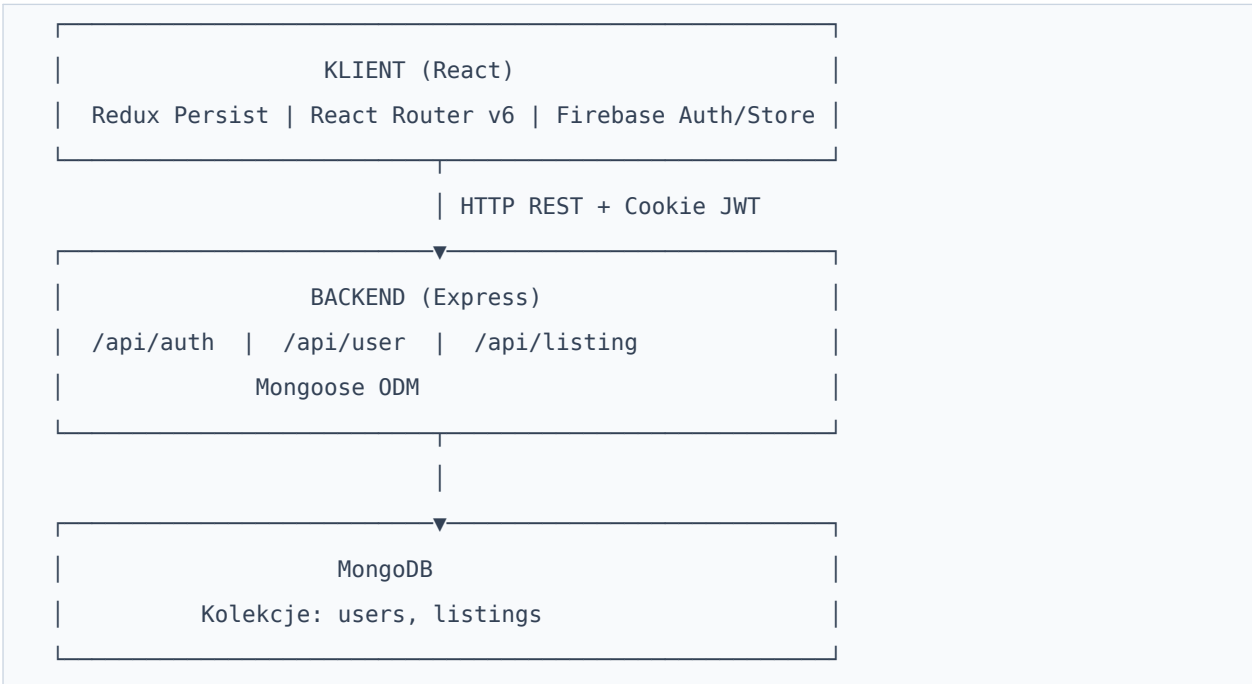
3. Struktura projektu

```
15-real-estate-moderate/
├─ api/
│   └─ controllers/
│       │   └─ auth.controller.js      # Rejestracja, logowanie, Google OAuth
│       │   └─ user.controller.js      # Zarządzanie użytkownikami
│       │   └─ listing.controller.js   # Ogłoszenia + wyszukiwanie
│       └─ models/
│           │   └─ user.model.js        # Model użytkownika (Mongoose)
│           │   └─ listing.model.js     # Model ogłoszenia (Mongoose)
│       └─ routes/
│           │   └─ auth.route.js
│           │   └─ user.route.js
│           │   └─ listing.route.js
│       └─ utils/
│           │   └─ error.js             # Pomocnik błędów HTTP
│           │   └─ verifyUser.js        # Middleware JWT
│       └─ index.js                    # Punkt wejścia backendu
├─ client/
│   └─ src/
│       │   └─ components/
│       │       │   └─ Header.jsx      # Nagłówek z wyszukiwarką
│       │       │   └─ ListingItem.jsx # Karta ogłoszenia
│       │       │   └─ Contact.jsx      # Kontakt z właścicielem
│       │       │   └─ OAuth.jsx        # Logowanie przez Google
│       │       │   └─ PrivateRoute.jsx # Ochrona tras prywatnych
│       │       └─ pages/
│       │           │   └─ Home.jsx / Search.jsx / Listing.jsx
│       │           │   └─ Profile.jsx / CreateListing.jsx / UpdateListing.jsx
│       │           │   └─ SignIn.jsx / SignUp.jsx / About.jsx
│       │           └─ redux/
│       │               │   └─ store.js
│       │               │   └─ user/userSlice.js
│       │           └─ firebase.js
│       └─ Dockerfile
└─ package.json
```

4. Architektura aplikacji

Przepływ danych

Aplikacja składa się z dwóch warstw komunikujących się przez HTTP REST. Frontend (React) wysyła żądania fetch do backendu (Express), który przetwarza je i operuje na bazie MongoDB przez Mongoose.



Przepływ autoryzacji

- Użytkownik przesyła dane logowania (email/hasło lub Google)
- Backend weryfikuje dane — porównuje hash bcrypt lub sprawdza e-mail Google
- Po weryfikacji generowany jest JWT i ustawiany jako cookie access_token z flagą httpOnly
- Każde chronione żądanie przechodzi przez middleware verifyToken, który weryfikuje JWT

5. Instalacja i uruchomienie

Wymagania wstępne

- Node.js w wersji 20 lub wyższej
- Instancja MongoDB (lokalna lub MongoDB Atlas)
- Projekt Firebase (autentykacja Google + Storage)

Kroki instalacji

```
# 1. Sklonuj repozytorium
git clone https://github.com/MaxGitHub0515/node-express-course-1.git
cd node-express-course-1/15-real-estate-moderate

# 2. Zainstaluj zależności backendu
npm install

# 3. Zainstaluj zależności frontendu
cd client && npm install && cd ..

# 4. Uruchom backend (port 3000)
npm run dev

# 5. Uruchom frontend w trybie dev (port 5173)
cd client && npm run dev
```

Wersja produkcyjna

```
npm run build  # buduje frontend do client/dist
npm start     # uruchamia serwer na porcie 3000
```

6. Zmienne środowiskowe

Utwórz plik `.env` w katalogu głównym projektu:

```
# Połączenie z MongoDB
MONGO=mongodb+srv://<użytkownik>:<hasło>@cluster.mongodb.net/estateflow

# Sekret do podpisywania tokenów JWT
JWT_SECRET=twoj_tajny_klucz_jwt
```

Konfiguracja Firebase (klucze API) znajduje się w pliku `client/src/firebase.js`. W środowisku produkcyjnym należy przenieść te wartości do zmiennych środowiskowych Vite (pliki `.env` z prefiksem `VITE_`).

7. API — dokumentacja endpointów

Autoryzacja — /api/auth

Metoda + Endpoint	Opis
POST /signup	Rejestracja nowego użytkownika
POST /signin	Logowanie e-mailem i hasłem
POST /google	Logowanie lub rejestracja przez Google OAuth
GET /signout	Wylogowanie — usuwa cookie access_token

Użytkownicy — /api/user (wymagana autoryzacja)

Metoda + Endpoint	Opis
POST /update/:id	Aktualizacja danych użytkownika
DELETE /delete/:id	Usunięcie konta i wyczyszczenie cookie
GET /listings/:id	Pobranie ogłoszeń należących do użytkownika
GET /:id	Pobranie publicznego profilu użytkownika

Ogłoszenia — /api/listing

Metoda + Endpoint	Auth	Opis
POST /create	Tak	Tworzenie nowego ogłoszenia
POST /update/:id	Tak	Aktualizacja istniejącego ogłoszenia
DELETE /delete/:id	Tak	Usunięcie ogłoszenia
GET /get/:id	Nie	Pobranie pojedynczego ogłoszenia
GET /get	Nie	Wyszukiwanie i filtrowanie ogłoszeń

Parametry wyszukiwania (GET /api/listing/get)

Parametr	Typ	Domyślnie	Opis
searchTerm	string	"	Szukaj po nazwie (regex)
type	string	all	Typ: sale rent all
offer	boolean	dowolny	Filtr ofert specjalnych
parking	boolean	dowolny	Filtr miejsc parkingowych
furnished	boolean	dowolny	Filtr umeblowania
sort	string	createdAt	Pole sortowania
order	string	desc	Kierunek: asc desc
limit	number	9	Wyniki na stronę
startIndex	number	0	Przesunięcie paginacji

8. Frontend — opis stron i komponentów

Home.jsx — Strona główna

Pobiera trzy zestawy ogłoszeń: oferty specjalne, wynajem i sprzedaż, każdy z limitem 4 wyników. Renderuje karuzelę Swiper oraz trzy sekcje siatki ogłoszeń z linkami "Pokaż więcej".

Search.jsx — Wyszukiwarka

Synchronizuje stan filtrów z parametrami URL (URLSearchParams). Przy każdej zmianie adresu URL pobiera nową listę z API. Przycisk "Pokaż więcej" dołącza kolejną stronę wyników.

Listing.jsx — Szczegóły ogłoszenia

Pobiera ogłoszenie po ID z URL. Renderuje karuzelę Swiper, przycisk kopiowania linku i formularz kontaktowy z właścicielem (dla zalogowanych użytkowników niebędących właścicielem).

Profile.jsx — Profil użytkownika

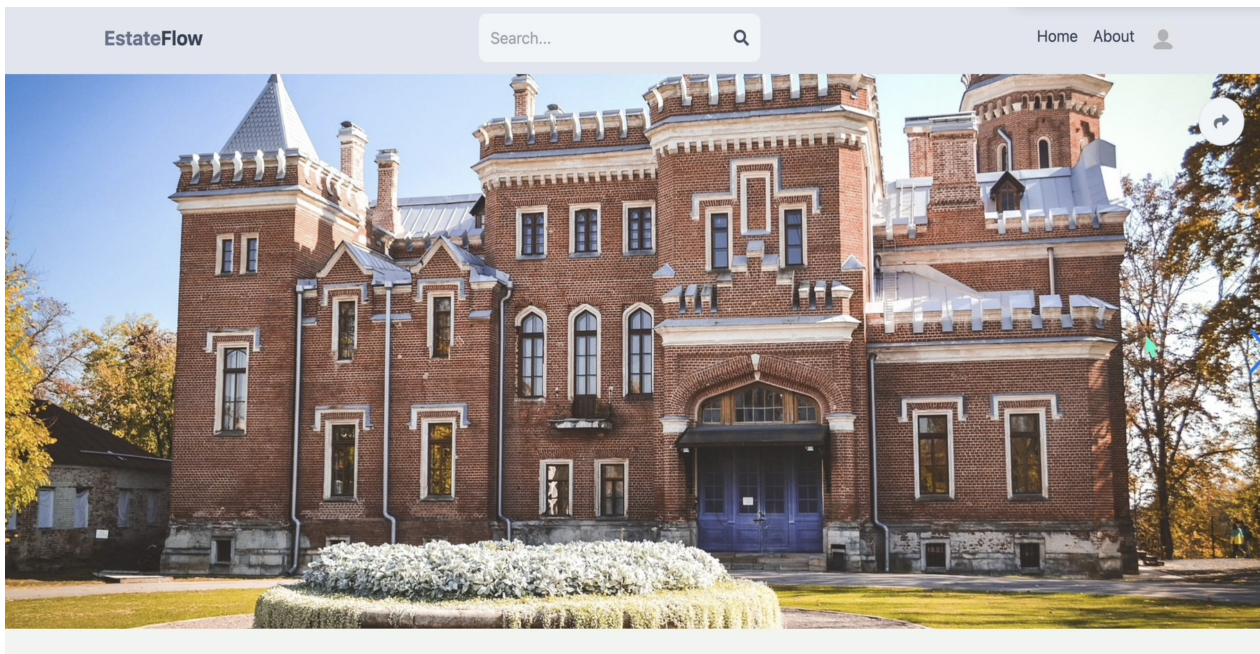
Obsługuje upload awatara do Firebase Storage z paskiem postępu, edycję danych konta, listę własnych ogłoszeń z opcją usunięcia lub edycji oraz wylogowanie i usunięcie konta.

CreateListing.jsx / UpdateListing.jsx — Formularze ogłoszeń

Identyczna struktura. UpdateListing pobiera przy montowaniu istniejące dane. Obie obsługują równoległy upload wielu zdjęć do Firebase Storage przez Promise.all.

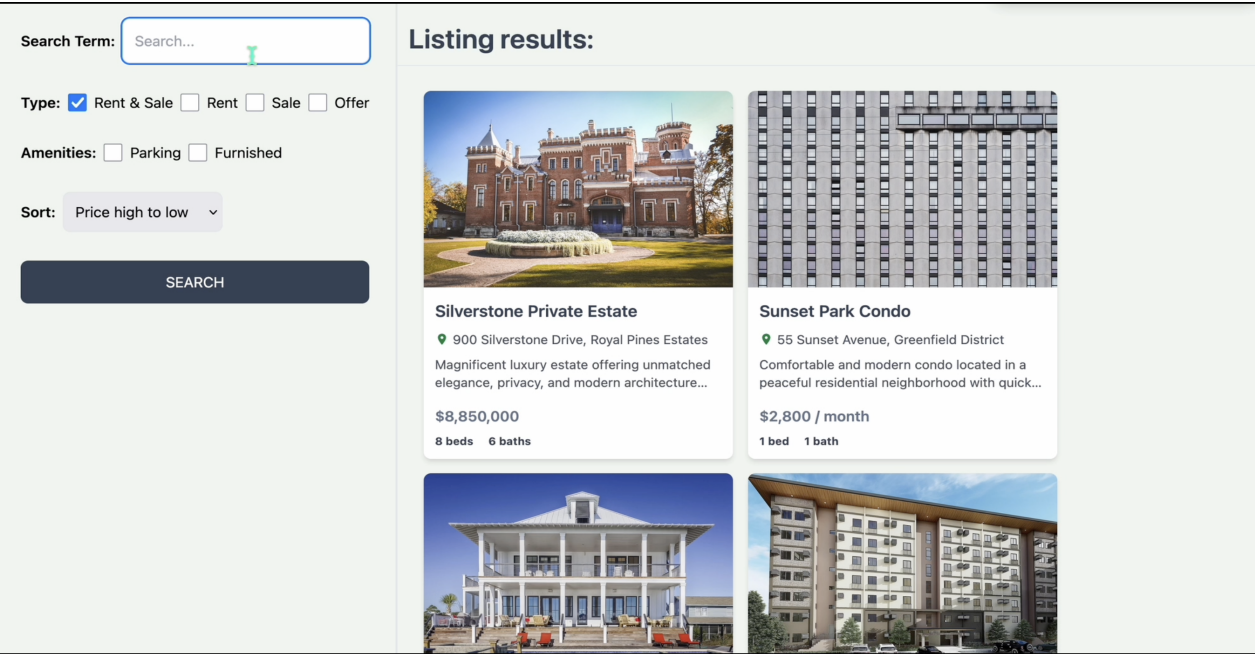
9. Podgląd aplikacji

Strona szczegółów ogłoszenia



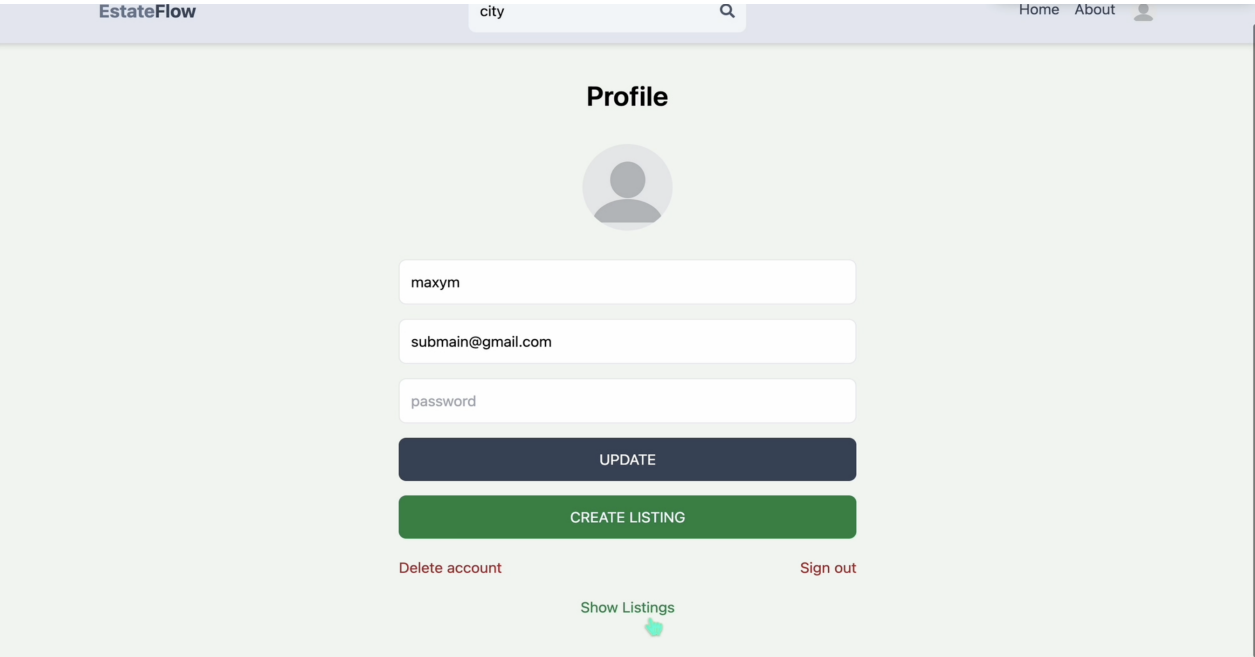
Rys. 1 — Pełnoekranowa karuzela zdjęć z detalami nieruchomości i ceną

Wyszukiwarka ogłoszeń



Rys. 2 — Panel filtrów z wynikami wyszukiwania i stronicowaniem

Profil użytkownika



Rys. 3 — Panel profilu z edycją danych i zarządzaniem ogłoszeniami

Tworzenie ogłoszenia

EstateFlow

Search...

Home About

Create a Listing

Silverstone Private Estate

gated entry. An exceptional opportunity for luxury living in one of the region's most prestigious communities.

900 Silverstone Drive, Royal Pines Estates

☒ Sell ☐ Rent ☒ Parking spot ☒ Furnished

☐ Offer

8

 Beds

6

 Baths

8900000

 Regular price

Images: The first image will be the cover (max 6)

Browse...

No files selected.

UPLOAD

CREATE LISTING

Rys. 4 — Formularz tworzenia ogłoszenia z przesyłaniem zdjęć

Edycja ogłoszenia

EstateFlow

city

Home About

Update a Listing

Emerald Hills Luxury Estate

An extraordinary luxury estate located in the exclusive Emerald Hills private community. This breathtaking property features grand modern architecture.

1 Emerald Crest Drive, Beverly Highlands

☒ Sell ☐ Rent ☒ Parking spot ☒ Furnished

☐ Offer

5

 Beds

4

 Baths

1250000

 Regular price

Images: The first image will be the cover (max 6)

Browse...

No files selected.

UPLOAD

 DELETE

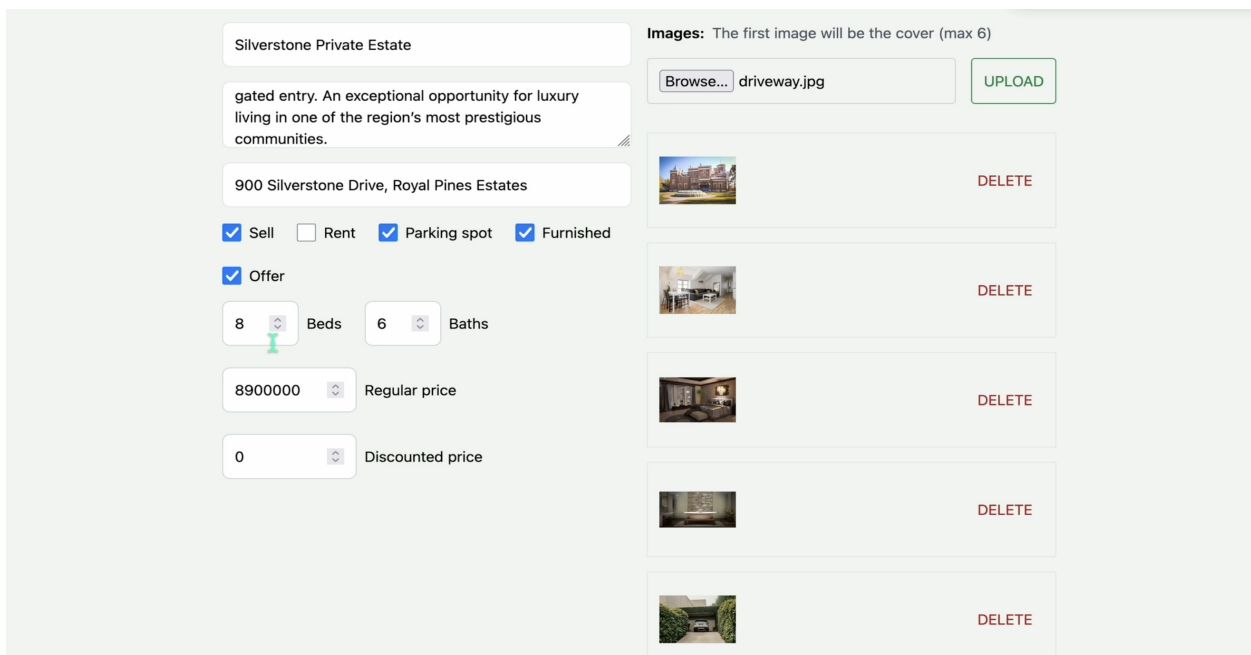
 DELETE

 DELETE

 DELETE

Rys. 5 — Formularz edycji ogłoszenia wstępnie wypełniony danymi

Zarządzanie zdjęciami



Silverstone Private Estate

gated entry. An exceptional opportunity for luxury living in one of the region's most prestigious communities.

900 Silverstone Drive, Royal Pines Estates

☒ Sell ☐ Rent ☒ Parking spot ☒ Furnished

☒ Offer

8 Beds 6 Baths

8900000 Regular price

0 Discounted price

Images: The first image will be the cover (max 6)

Browse... driveway.jpg UPLOAD

DELETE

DELETE

DELETE

DELETE

DELETE

Rys. 6 — Panel zdjęć z podglądem miniatur i usuwaniem plików

10. Autoryzacja i bezpieczeństwo

JWT i ciasteczka HTTP-only

Po zalogowaniu serwer generuje token JWT i ustawia go jako ciasteczko httpOnly, co uniemożliwia dostęp przez JavaScript i chroni przed atakami XSS.

```
res.cookie('access_token', token, { httpOnly: true })
```

Hashowanie haseł

Hasła są hashowane przy użyciu bcryptjs z czynnikiem 10 przed zapisem do bazy:

```
const hashedPassword = bcryptjs.hashSync(password, 10);
```

Kontrola dostępu do zasobów

- Modyfikacja własnego konta: `req.user.id === req.params.id`
- Modyfikacja własnego ogłoszenia: `req.user.id === listing.userRef`

11. Zarządzanie stanem (Redux)

Aplikacja używa Redux Toolkit do zarządzania stanem użytkownika oraz Redux Persist do zachowania sesji w localStorage między odświeżeniami strony.

Akcja	Opis
signInStart / Success / Failure	Logowanie — loading, zapis danych, błąd
updateUserStart / Success / Failure	Aktualizacja profilu
deleteUserStart / Success / Failure	Usunięcie konta — zeruje currentUser
signOutUserStart / Success / Failure	Wylogowanie — zeruje currentUser

12. Docker i wdrożenie

Serwer Express w trybie produkcyjnym serwuje zbudowaną aplikację React na tym samym porcie 3000 — API pod /api/* i frontend pod pozostałymi ścieżkami.

```
# Zbuduj obraz
docker build -t estateflow .

# Uruchom kontener
docker run -p 3000:3000 \
  -e MONGO="mongodb+srv://..." \
  -e JWT_SECRET="twój_sekret" \
  estateflow
```

13. Znane problemy

Problem	Plik	Opis
Błędna nazwa pola sort	Search.jsx	'created_at' zamiast 'createdAt' — sortowanie nie działa przy zał...
Brak console.	Home.jsx	log(error) zamiast console.log(error) — ReferenceError przy błę...
Brak wygaśnięcia JWT	auth.controller.js	Token JWT bez expiresIn — sesje nie wygasają po stronie serwer...
Klucze Firebase w kodzie	firebase.js	API key w pliku źródłowym — przenieść do zmiennych środowisk...

Pusty client/Dockerfile	client/Dockerfile	Plik Dockerfile dla frontendu jest pusty
-------------------------	-------------------	--